

# Financial Monthly: Capstone 3

Author: Logan Grace

Master of Science in Software Engineering Capstone Design Specification

Grand Canyon University

Instructor: Professor Hatim Alzoubi

Revision: -

Date: 11-11-2026

© 2026. Grand Canyon University. All Rights Reserved.

# ABSTRACT

Financial monthly is a project that applies simple concepts into a complex software program. The program will be web based and allow the user to get a better understanding of someone's personal finances. The project is simple; it has the user input their financial information by simple prompts. From there the program will take that information and run back-end calculations to showcase the projections. The questions being asked are monthly rent, grocery prices, 401K contribution, gasoline, fun purchases, vacations, and many more options that can be added at the user's discretion.

That on its own is not too unique. What makes it unique is all the functions that can be used with that information. When the user completes their part, the program will compile the information and display different numbers. It could be emergency fund recommendation, how your bills are allocated by importance, how your bills fit common financial rules, how your retirement may look when you want to retire, and many more. It will even have the option to display a pdf or excel version to be able to have a physical copy of the results when needed.

**Summary of Work**

Most of the foundation for this project is complete with the architecture and the routing. The general user stories are input and have been adapted to the program. More is still to come as financial information is endless and can be applied anywhere. One area I want to complete before the final submission creating the login page to begin storing the user data. This will allow the user to have the information saved if they wish to return to the program later for adjustments.

## HISTORY AND SIGNOFF SHEET

### Change Record

| Date | Author | Revision Notes |
|---|---|---|
| 11/11/26 | Logan Grace | Initial draft for review/discussion |
| | | |
| | | |

**Overall Instructor Feedback/Comments**

**Overall Instructor Feedback/Comments**

**Integrated Instructor Feedback into Project Documentation**

☐ Yes ☐ No

**Project Approval**

☐ *<Insert name of instructor here>*

# TABLE OF CONTENTS

**DESIGN INTRODUCTION** ..... **4**
**DETAILED HIGH-LEVEL SOLUTION DESIGN** ..... **5**
**DETAILED TECHNICAL DESIGN** ..... **8**
**APPENDIX A – TECHNICAL ISSUE AND RISK LOG** ..... **16**
**APPENDIX B – REFERENCES** ..... **17**
**APPENDIX C – COPYRIGHT COMPLIANCE** ..... **18**
**APPENDIX D – EXTERNAL RESOURCES** ..... **19**

## Design Introduction

Financial Monthly is designed to provide users with a simple way to manage their monthly finances and better understand their long-term financial future. The solution is a web-based application that allows users to enter their monthly income, organize expenses by category, and evaluate how their current financial decisions may affect their future finances The overall design focuses on creating an intuitive user experience while keeping the application's underlying architecture maintainable

The application uses a React frontend to provide the user interface and a Python FastAPI backend to handle financial calculations and process submitted information. The frontend collects information such as monthly income and expenses for the calculations. This information is sent to the backend through API requests, where the application calculates monthly overview and projections for the future. The results are then returned to the front end and presented in an easy visual so the user can get a better understanding without getting confused.

The design also emphasizes separation of responsibilities between the user interface, backend services, and calculation logic. This approach makes the application easier to maintain and allows individual features to be improved without requiring significant changes to the entire system. Future enhancements may include user authentication, saving financial information between sessions, and exporting financial reports to PDF or Excel.

Because this is just a project rather than a business application for now, there is no external stakeholder responsible for approving the design. The design concepts are evaluated against the project's original objectives and user experience. The proposed design is satisfactory because it addresses the application's primary purpose of helping individuals understand their monthly finances and make more informed financial decisions.

**Project Deliverables:**

- Data Dictionary
- API Design Documentation
- Database Design
- User Interface Design
- Logical Architecture Diagram
- Financial Calculation Documentation
- Test Plan and Test Cases
- Installation and User Documentation

## Detailed High-Level Solution Design

### Introduction

This project is designed to give users a simple way to manage their monthly finances and plan. The application allows users to enter their income and expenses, calculate potential savings, and explore financial goals such as emergency funds and retirement planning. The design focuses on keeping the application easy to use while organizing the frontend, backend, and calculations so they can be maintained and improved as the project grows.

### Detailed Overview

This project uses three-layer architecture consisting of the React, Python FastAPI, and financial calculation. The frontend collects user information and displays results, while the backend processes requests and does the math. The logical design shows how these components communicate, while the physical design illustrates how the frontend and backend run as separate services.

Some configuration is required to connect the frontend and backend, including API endpoint settings, CORS permissions, and environment variables. Security will be supported through various methods but mainly separating the frontend from thee backend. Performance will be maintained through efficient calculations and lightweight API requests.

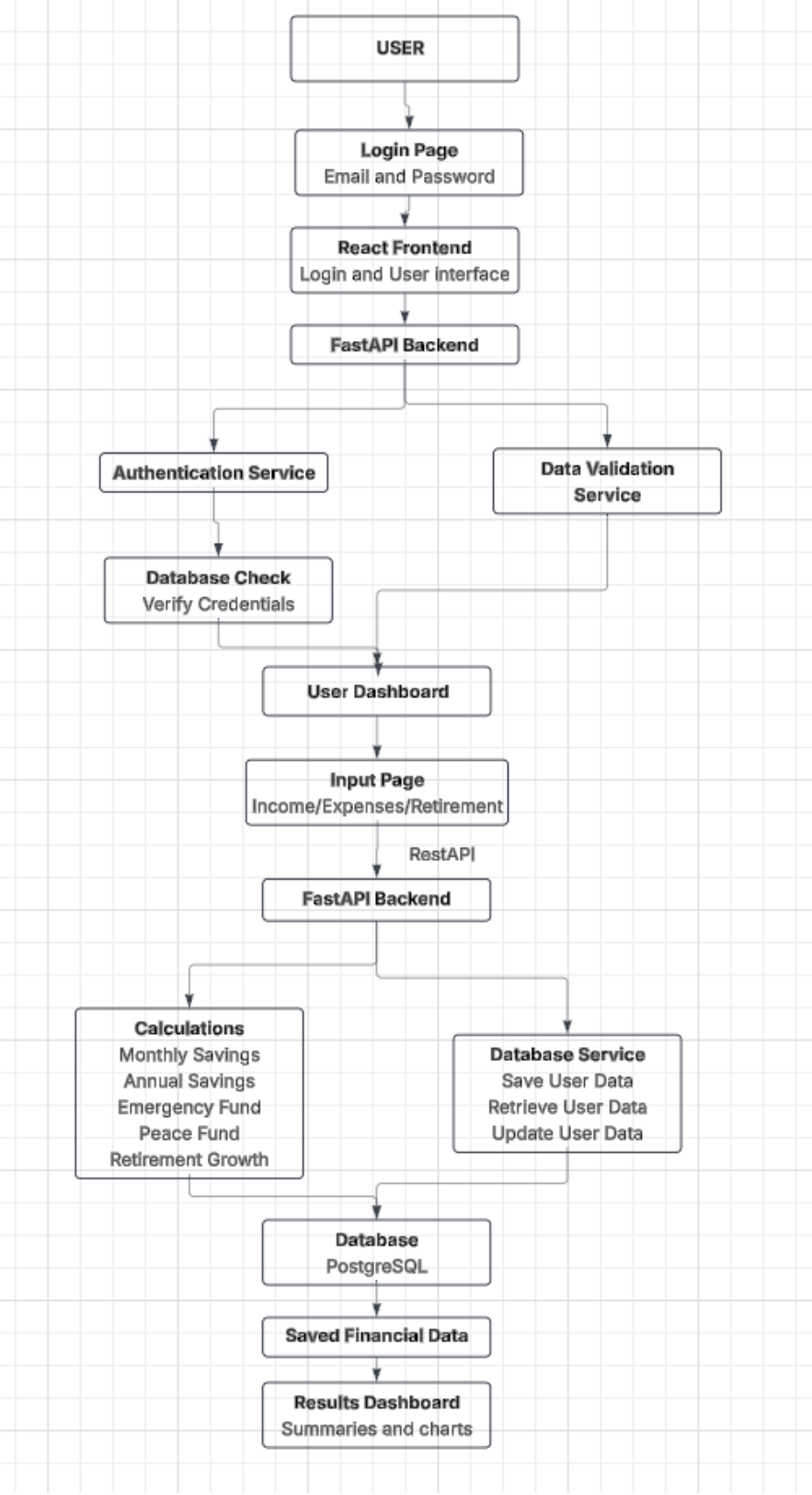
The purpose of this detailed architecture is to give developers a clear understanding of how the application is organized, how its components interact, and what is needed to build, maintain, and expand the system.

### Hardware and Software Technologies

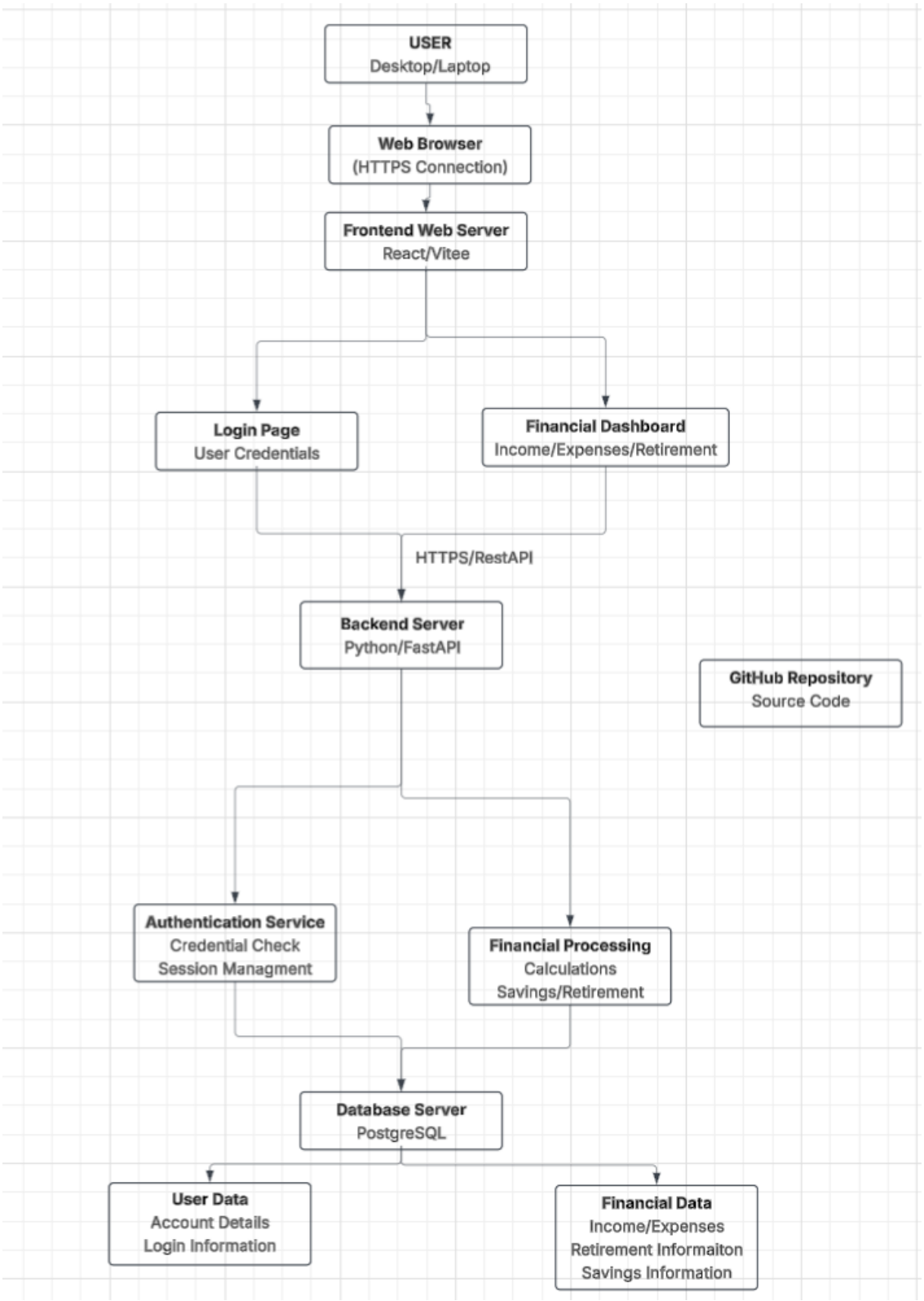
| HARDWARE AND SOFTWARE TECHNOLOGIES |
|---|
| 1. Standard Desktop with all components needed |
| 2. VS Code |
| 3. Git and GitHub |
| 4. Postman |
| 5. HTML/CSS |
| 6. Pydantic |
| 7. Uvicorn |
| 8. FastAPI |
| 9. JavaScript |
| 10. Python |
| 11. React.js |

| PROOF OF CONCEPTS | | |
|---|---|---|
| **Description** | **Rationale** | **Results** |
| 1. React (Frontend) | Verify that React can support an interactive and user-friendly interface for entering all inputs | Working. Supports dynamic forms and displaying results |
| 2. FastAPI (Backend) | Confirm that Fast API can receive user information and process calculations for a displayed result | Working. Fast API process request and returns accurate calculations for the user to see. |
| 3. Integration for Front and Back | Proper communication between the front and back end | Working. Frontend can send information to backend and receive results. |
| 4. Calculation Logic | Confirm that the application can accurately calculate all results implemented in the frontend | Working. Backend can produce accurate results confirmed by calculation check. |
| 5. Data Visualization | Implement charts where it makes sense to give the user a visual experience instead of just quantitive | Working. Have pie charts and graphs to help with simple reviews |
| 6. GitHub Version Control | Confirm GitHub can maintain and update as more features are added | Working. GitHub repository is able to keep up with changes. |

## Logical Solution Design



## Physical Solution Design



# Detailed Technical Design

## General Technical Approach

Financial Monthly will be developed as a web-based application using React for the frontend and Python with FastAPI for the backend. The application will allow users to create an account, log in, enter their monthly income and expenses, and view financial projections. The backend will handle data validation, financial calculations, and communication with a PostgreSQL database to securely store user information. The database will store any information the user has input for future applications. The overall approach is to keep the application simple and easy to use while keeping the code organized so that additional features can be added as the project grows.

## Key Technical Design Decisions

The application uses React for the frontend because it supports reusable components and an interactive user interface. Python with FastAPI was selected for the backend because it is lightweight, efficient, and supports REST API development. JavaScript, HTML, and CSS support frontend functionality and styling. These were chosen because they are relatively easy to maintain and support any future additions. Proof of concept was completed by connecting the React frontend to the FastAPI backend and successfully submitting financial information to generate savings and retirement calculations. This validated that the selected technologies could support the application's primary requirements

## Database ER Diagram

Will provide when flow is in proper place for all data

## Database Dictionary

| Description | Datatype | Business Rules | Length | Default Value | Nulls (Y/N) | PK (Y/N) | FK (PK Column) | Comment |
|---|---|---|---|---|---|---|---|---|
| Retirement ID | INTEGER | Unique identifier for each retirement plan | N/A | Auto-generated | N | Y | N/A | Primary key |
| User ID | INTEGER | Must reference an existing user | N/A | None | N | N | USERS (user_id) | Foreign key |
| Monthly Contribution | DECIMAL | Must be greater than or equal to zero | 10,2 | None | N | N | N/A | Monthly retirement contribution |
| Annual Return Rate | DECIMAL | Must be a valid percentage | 5,2 | None | N | N | N/A | Expected annual return |
| Current Age | INTEGER | Must be a positive number | 3 | None | N | N | N/A | User's current age |
| Retirement Age | INTEGER | Must be greater than current age | 3 | None | N | N | N/A | Target retirement age |

| Description | Datatype | Business Rules | Length | Default Value | Nulls (Y/N) | PK (Y/N) | FK (PK Column) | Comment |
|---|---|---|---|---|---|---|---|---|
| Expense ID | INTEGER | Unique identifier for each expense | N/A | Auto-generated | N | Y | N/A | Primary key |
| Record ID | INTEGER | Must reference an existing financial record | N/A | None | N | N | MONTHLY_FINANCIAL_RECORDS (record_id) | Foreign key |
| Expense Name | VARCHAR | Cannot be empty | 100 | None | N | N | N/A | Name of expense |
| Category | VARCHAR | Must identify an expense category | 50 | None | N | N | N/A | Expense classification |
| Amount | DECIMAL | Must be greater than or equal to zero | 10,2 | None | N | N | N/A | Expense amount |

| Datatype | Business Rules | Length | Default Value | Nulls ( | PK (\ | FK (F | Comment |
|---|---|---|---|---|---|---|---|
| INTEGER | Unique identifier for each user | N/A | Auto-generated | N | Y | N/A | Primary key |
| VARCHAR | Must be unique | 50 | None | N | N | N/A | User's login name |
| VARCHAR | Must be unique and valid | 255 | None | N | N | N/A | User's email address |
| VARCHAR | Must contain securely hashed password | 255 | None | N | N | N/A | Stores hashed credentials |
| TIMESTAMP | Automatically records account creation | N/A | CURRENT_TIMESTAMP | N | N | N/A | Account creation date |

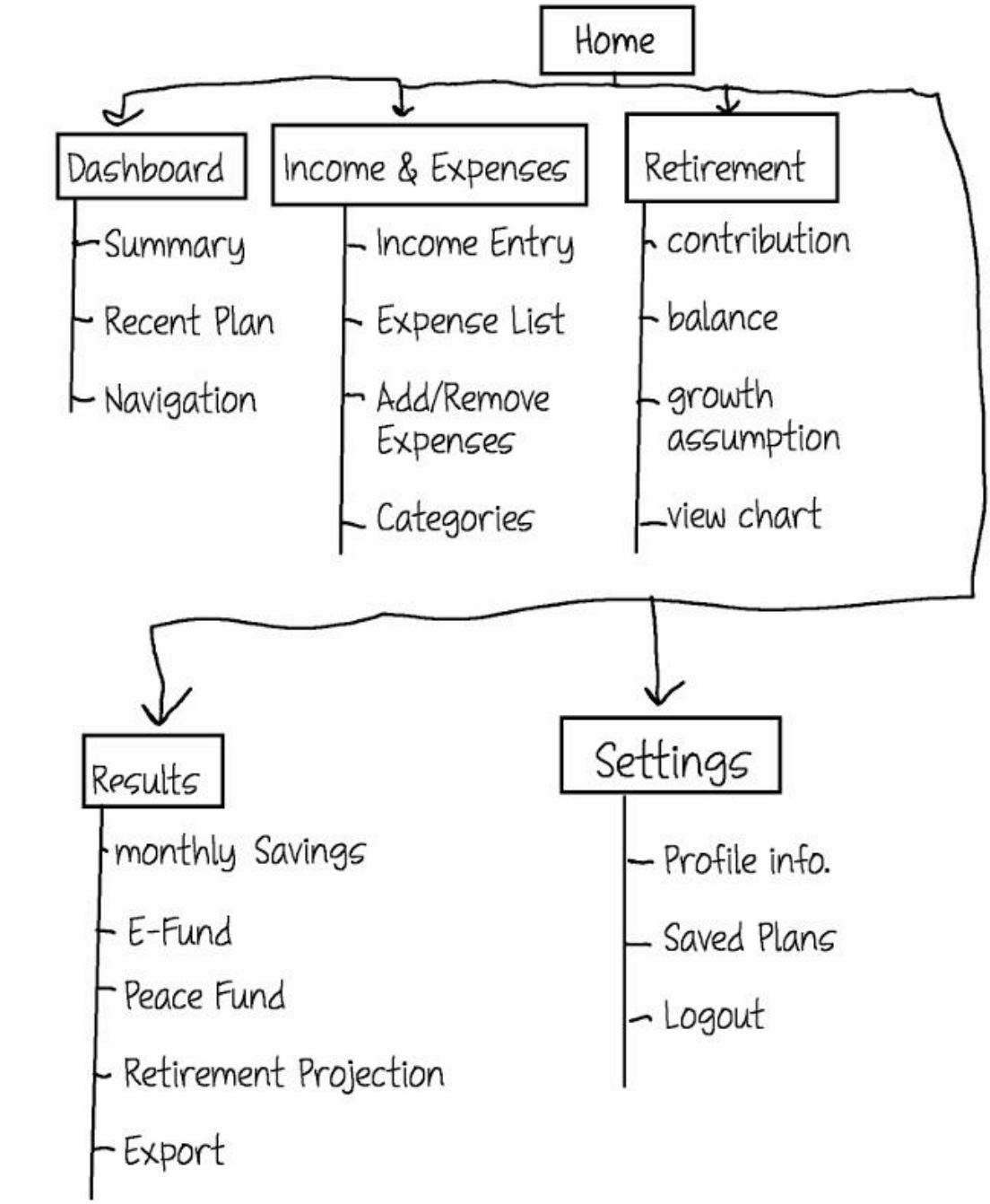
| Description | Datatype | Business Rules | Length | Default Value | Nulls (Y/N) | PK (Y/N) | FK (PK Column) | Comment |
|---|---|---|---|---|---|---|---|---|
| Record ID | INTEGER | Unique identifier for each record | N/A | Auto-generated | N | Y | N/A | Primary key |
| User ID | INTEGER | Must reference an existing user | N/A | None | N | N | USERS (user_id) | Foreign key |
| Monthly Income | DECIMAL | Must be greater than or equal to zero | 10,2 | None | N | N | N/A | Monthly income amount |
| Month | INTEGER | Must be between 1 and 12 | 2 | None | N | N | N/A | Financial record month |
| Year | INTEGER | Must represent a valid year | 4 | None | N | N | N/A | Financial record year |
| Created At | TIMESTAMP | Automatically records creation date | N/A | CURRENT_TIMESTAMP | N | N | N/A | Record creation date |

**Database DDL Scripts**

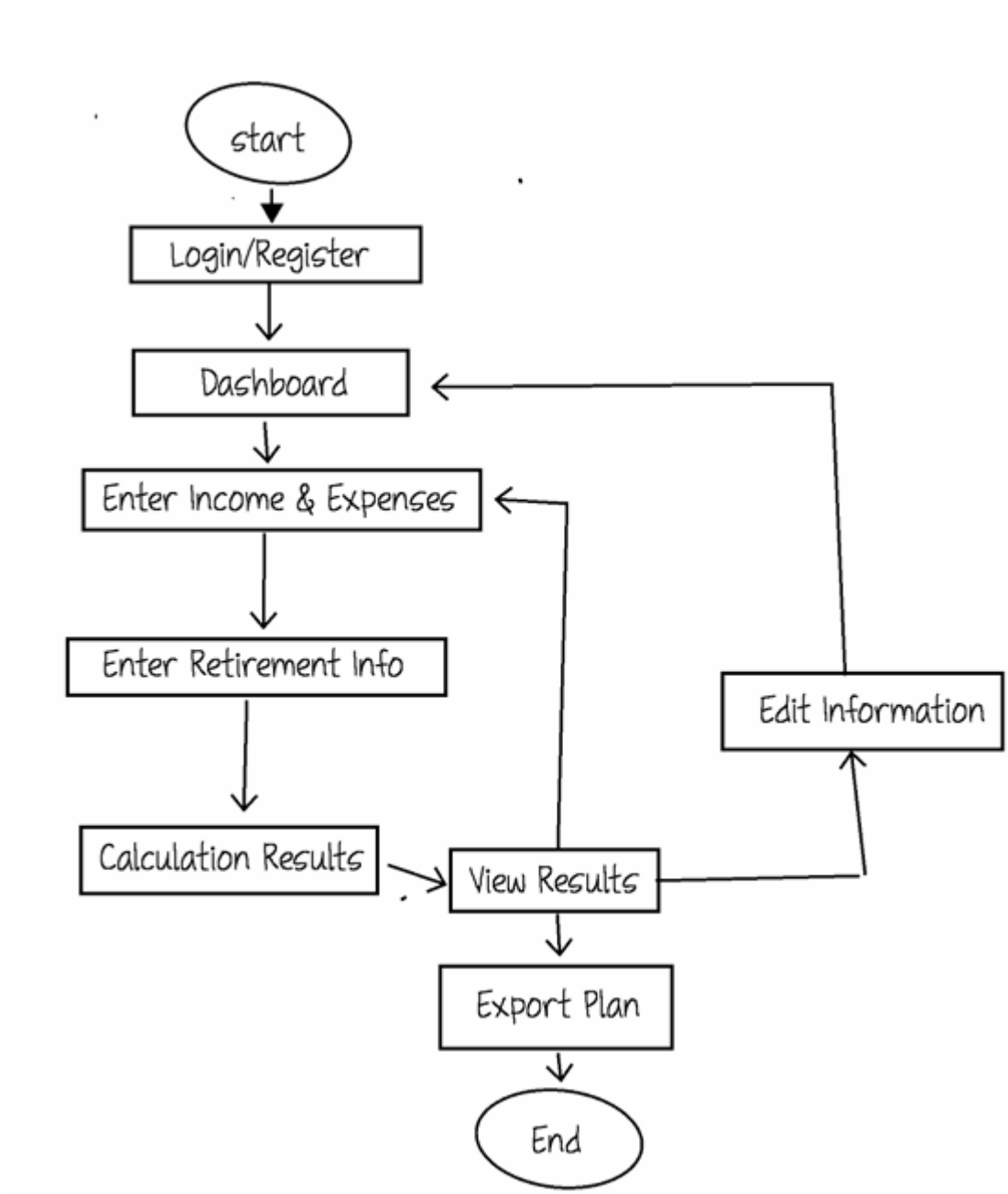
```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
```

Will be adding more when the financial data is in a saved position.

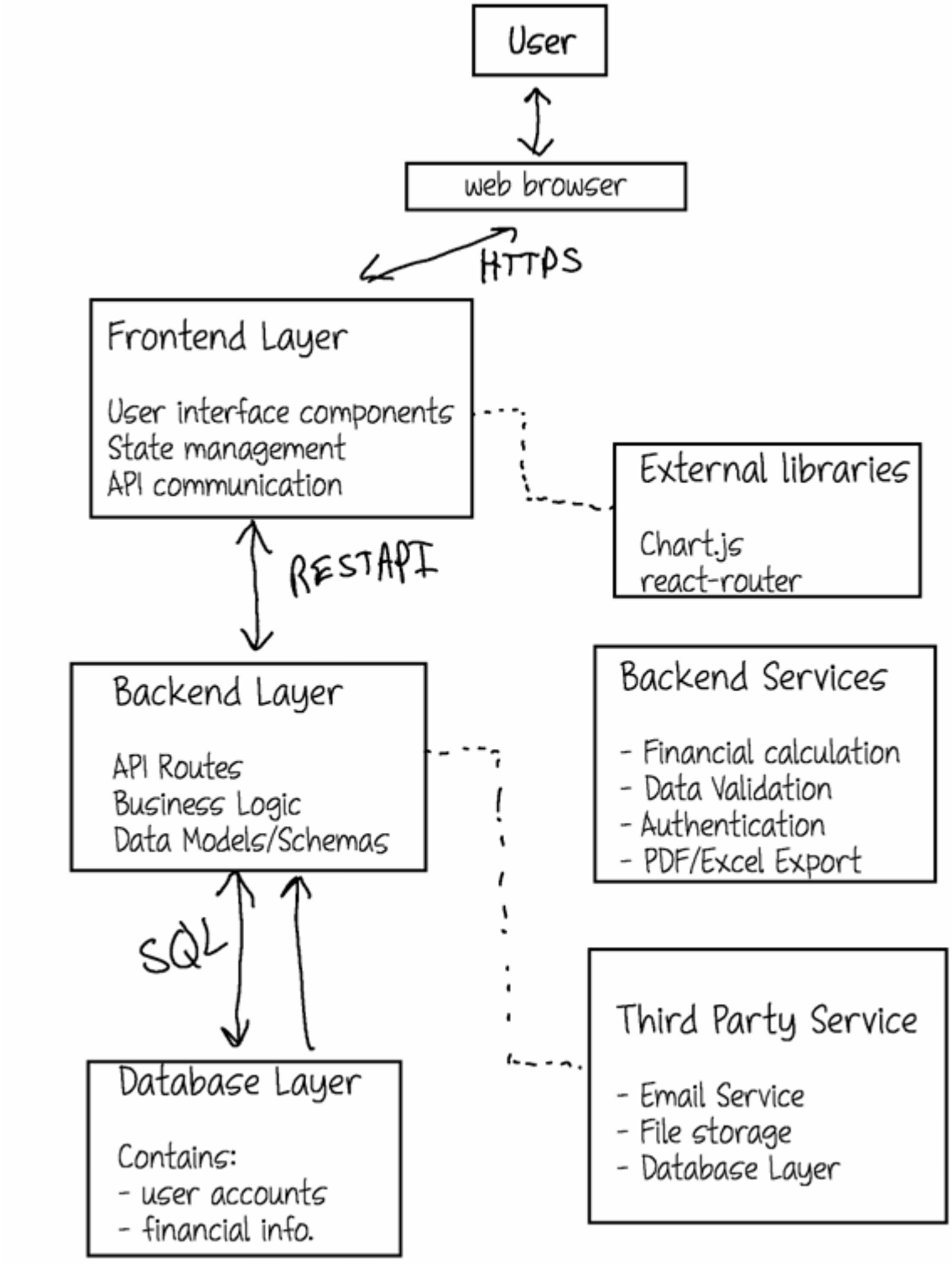
**Flow Charts/Process Flows:**



**Sitemap Diagram**

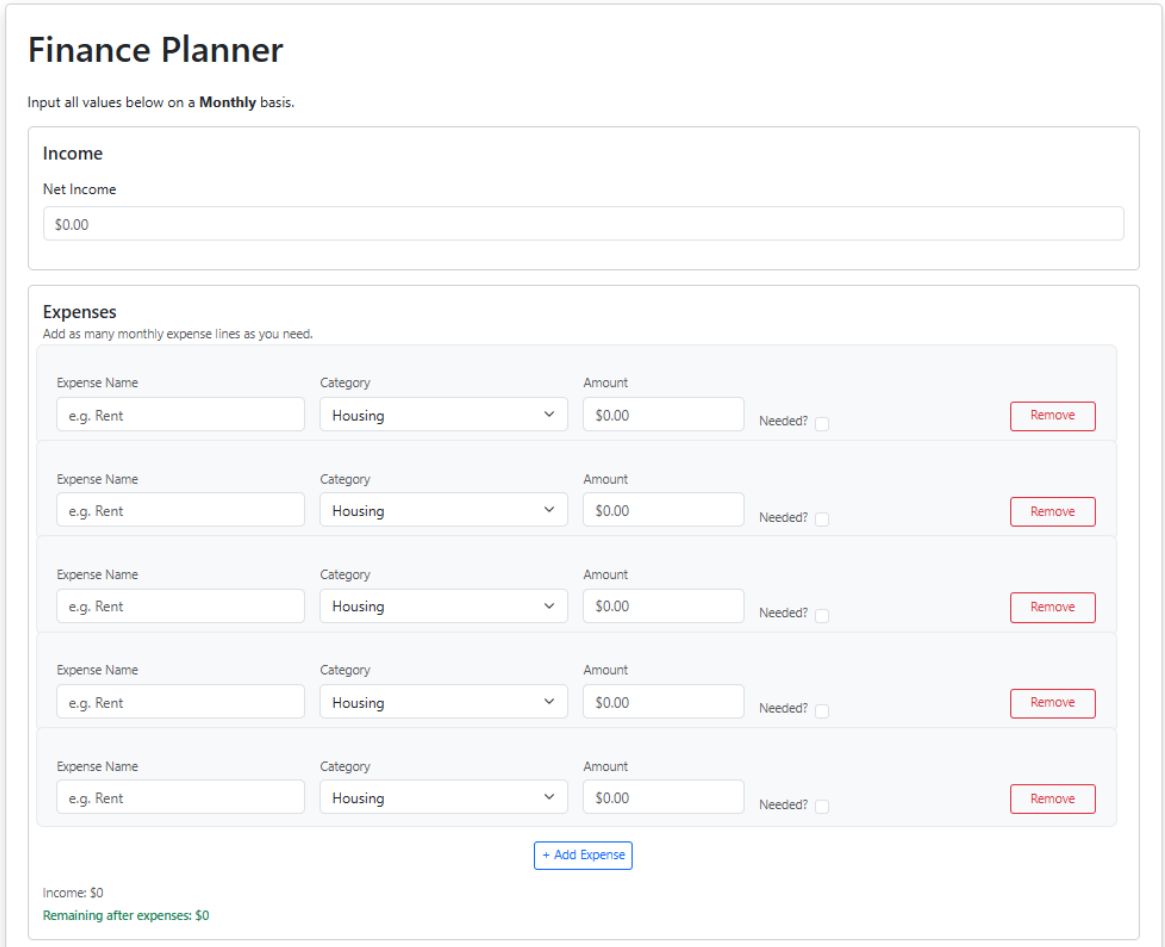


**User Interface Diagrams**



*Screen Definitions and Layouts*



**Service API Design**

The application publishes an application-specific REST API using Python FastAPI. The primary endpoint is POST /api/calculate, which accepts financial information from the React frontend and returns calculated financial results.

**NFR's (Security Design, etc.)**

This will support non-functional requirements by focusing on security and reliability. React will provide a simple and responsive interface, while FastAPI will handle requests and financial calculations efficiently. PostgreSQL will securely store user accounts and financial information, with authentication and data validation helping protect sensitive information. The application will also be designed with organized, reusable components so future updates and additional features can be added without significantly changing the existing structure.

*Security*

Biggest security risk is access to the database. There is no real harm in the general system itself. The database does store emails and passwords for the user to return with their financial information already in the system. Another risk is that the user is putting in financial information. The system does not ask for bank accounts or retirement accounts. It does ask for what they are contributing and what they are spending money on. Not the biggest risk but should be handled with the username and password security.

**Operational Support Design**

It will support monitoring and logging through its frontend and backend components. The FastAPI backend can record important events like user log in and errors in calculation. These logs will help identify issues and make troubleshooting easier. The application will also monitor data validation and database operations to help ensure information is processed correctly.

## Appendix A – Technical Issue and Risk Log

[Identify and monitor project issues and risks.]

| ISSUES AND RISK LOG | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Issue or Risk** | **Description** | **Project Impact** | **Action Plan/Resolution** | **Owner** | **Importance** | **Date Entered** | **Date to Review** | **Date Resolved** |
| I/R | What is the issue or risk? | How will this impact scope, schedule, and cost? | How do you intend to deal with this issue? | Who manages this issue? | | | | |
| | | | | | | | | |
| | | | | | | | | |

## Appendix B – References

## Appendix C – Copyright Compliance

Ecma International. (n.d.). *ECMAScript language specification*. https://tc39.es/ecma262/

FastAPI. (n.d.). *FastAPI license*. https://github.com/fastapi/fastapi/blob/master/LICENSE

Git. (n.d.). *Git license*. https://github.com/git/git/blob/master/COPYING

GitHub. (n.d.). *GitHub terms of service*. https://docs.github.com/en/site-policy/github-terms/github-terms-of-service

Microsoft. (n.d.). *Visual Studio Code license*. https://code.visualstudio.com/license

PostgreSQL Global Development Group. (n.d.). *PostgreSQL license*. https://www.postgresql.org/about/licence/

Postman. (n.d.). *Terms of service*. https://www.postman.com/legal/terms/

Pydantic. (n.d.). *Pydantic license*. https://github.com/pydantic/pydantic/blob/main/LICENSE

Python Software Foundation. (n.d.). *Python license*. https://docs.python.org/3/license.html

React. (n.d.). *React license*. https://github.com/facebook/react/blob/main/LICENSE

Uvicorn. (n.d.). *Uvicorn license*. https://github.com/encode/uvicorn/blob/master/LICENSE.md

Vite. (n.d.). *Vite license*. https://github.com/vitejs/vite/blob/main/LICENSE

WHATWG. (n.d.). *HTML living standard*. https://html.spec.whatwg.org/

World Wide Web Consortium. (n.d.). *CSS specifications*. https://www.w3.org/Style/CSS/

## Appendix D – External Resources

| | |
|---|---|
| **GIT URL:** | *LGrace9784/Financial-Monthly* |
| **Hosting URL:** | *(if applicable).* |
| **Screencast URL:** | *(if applicable).* |